

Table of Contents

OTC Derivatives Compliance Engine — Technical and Regulatory Report.....	2
1. Regulatory Landscape.....	2
2. System Architecture.....	3
3. Portfolio Findings	4
4. Classification Analysis — Prediction Contracts (Module 4)	5
5. Limitations and Future Work.....	5
Team Contribution Statement	6
References	7

OTC Derivatives Compliance Engine — Technical and Regulatory Report

This report covers the engine’s regulatory context, system architecture, portfolio findings, classification analysis, and limitations.

1. Regulatory Landscape

The global OTC derivatives market had roughly USD 846 trillion in outstanding notional as of mid-2025 (BIS 2025). Following the 2008 financial crisis and the G20 Pittsburgh commitments, this market shifted from bilateral opacity to a regime of standardized trade reporting under CFTC Part 45 in the United States (CFTC 2012), the European Market Infrastructure Regulation (Regulation (EU) 648/2012) in the EU, and equivalent national frameworks in Singapore, Australia, Japan, Canada, and the UK. The post-crisis reporting architecture rests on three identifier pillars that the engine validates directly.

The **Unique Product Identifier (UPI, ISO 4914)** identifies the *type* of derivative product. ANNA-DSB has been the designated sole issuer since 2019 and maintains a library of 143 product definition templates across five asset classes (Rates, Credit, FX, Equity, Commodities). UPI reporting became mandatory in the US in January 2024, in the EU in April 2024, in the UK in September 2024, and in Australia, Singapore, and Japan by April 2025. Multiple trades of the same product type share the same UPI; the engine’s deterministic 12-character mock UPI (Module 2) preserves this property across the portfolio.

The **Unique Transaction Identifier (UTI, ISO 23897)** identifies a specific trade instance. Its format is a 20-character LEI namespace concatenated with a suffix of up to 32 uppercase alphanumeric or hyphen characters, capped at 52 characters total. Both counterparties to a bilateral trade must report the same UTI, which makes SDR-side pairing reconciliation a downstream concern beyond this engine’s single-side scope.

The **Legal Entity Identifier (LEI, ISO 17442)** identifies each legal entity. Issued by GLEIF and accredited Local Operating Units, it is a 20-character string of 18 alphanumerics plus 2 numeric check digits computed under ISO 7064 MOD 97-10. The engine implements this checksum (via `python-stdnum` plus a transparent `compute_lei_check_digits` helper) and applies it to every counterparty LEI in the portfolio. Three of the eight distinct LEIs in `trades.json` fail the checksum — the assignment’s “test LEIs are not all real” teaching point — and the engine surfaces these as compliance failures rather than silently accepting them.

The regimes converge on identifier formats but diverge on field requirements. The CPMI-IOSCO Critical Data Elements technical guidance (CPMI-IOSCO 2018) defines the common fields; jurisdictional rules then add overlays. The clearest divergence the engine encodes is collateral and margin reporting: CFTC Part 45 does not require

collateral_portfolio_code, initial_margin_posted, or variation_margin_posted, whereas MAS, EMIR Refit (Regulation (EU) 2019/834), and ASIC all do. This single divergence is responsible for most of the asymmetry between CFTC and MAS compliance counts in the portfolio.

2. System Architecture

The engine is a five-module pipeline. Each module reads and writes JSON and runs independently; later modules consume earlier modules' outputs, but each can be rerun in isolation when its inputs change. The pipeline order is

Module 1 (parser and instrument classifier)

Module 2 (UPI lookup against ANNA-DSB)

Module 3 (multi-jurisdictional compliance)

Module 4 is written analysis with no code. The total Python footprint is around 2,500 lines across four .py files, with two third-party dependencies (python-stdnum for LEI MOD 97-10 and pycountry for ISO 4217), plus Flask and Plotly for the dashboard.

Module 5 (Flask dashboard).

Several design decisions shaped the engine's behavior.

Strict identifier validation, no auto-aliasing. execution_timestamp is required to match YYYY-MM-DDTHH:MM:SSZ exactly — T013's date-only value "2025-09-01" is therefore rejected, not silently promoted to midnight UTC. maturity_date is parsed with datetime.strptime, so T021's syntactically-passing "9999-99-99" fails the calendar check. Reference rates absent from FpmlRatesReferenceRate.json (notably EUR-ESTR in T006/T011/T018 and GBP-RPI in T013) are surfaced as errors rather than aliased to the closest codeset entry. The engine's job is to surface data-quality issues, not paper over them.

Exact-then-heuristic template matching. ANNA-DSB filename stems rarely match trade use_case strings verbatim (Cap_Floor.Cap is not in the Rates folder; the matching template is Rates.Option.CapFloor.UPI.V1). The engine first attempts an exact match on (asset_class, instrument_type, use_case). Where that fails, a token-overlap scoring function splits both sides on underscores and camel-case boundaries, scores common × 10 – extra, and selects the highest-scoring template, with version preference. A subject hint derived from the trade payload (underlying_isin → single name; underlying_index → single index) disambiguates equity variants. The chosen template name is always recorded in classification_note, so heuristic matches are visible rather than hidden. The 28-trade portfolio yielded 11 exact and 14 heuristic matches.

Deterministic mock UPI. The cloned ANNA-DSB repository contains JSON Schemas, not records — the real UPI string is generated by the DSB live API at registration and is not extractable from the repo. The engine hashes (matched_template, UPI-defining attribute values) to SHA-256, base32-encodes the digest, and takes the first 12 characters. Trade-specific fields (notional_amount, dates, counterparty LEIs) are deliberately excluded from the hash so that two trades on the same product receive the same mock code — preserving UPI identity semantics.

Per-field audit trail. Module 3's field_validations block records each required field as {value, valid, error}, regardless of whether it passes or fails. This costs roughly 5× the output size compared to a failures-only schema, but means a marker (or future regulator) can confirm that the engine actually ran each check rather than inferring it from absence.

The hardest problems were taxonomic rather than technical: the use-case-name divergence above; multi-leg rate validation for cross-currency and basis swaps (T006, T011), where a single reference_rate becomes reference_rate_leg1 / reference_rate_leg2; the NOVEL-vs-CONVENTIONAL applicability rule for T026–T028, which decides regime status from platform_type before any field validation runs; the LIBOR substring override that downgrades a codeset-match failure to a warning for legacy trades; and the integration of Module 2's mock UPI as a proxy for UPI presence in Module 3, avoiding the alternative of marking every conventional trade NONCOMPLIANT for the same uninformative reason.

3. Portfolio Findings

The 34-trade portfolio (28 assignment-provided + 6 author-designed, per Deliverable 2) produced the following compliance distribution: CFTC — 4 COMPLIANT, 26 NONCOMPLIANT, 3 CONDITIONAL, 1 NOT_APPLICABLE; MAS — 3 COMPLIANT, 27 NONCOMPLIANT, 0 CONDITIONAL, 4 NOT_APPLICABLE. With only T029, T030, and T033 (author-designed clean trades) plus T017 (CFTC only) passing, the report is dominated by failure categories, which is exactly what makes the dataset pedagogically useful.

Three patterns drive the failure picture. First, **LEI checksum failures dominate**. Three of the eight distinct counterparty LEIs in the dataset fail ISO 7064 MOD 97-10 (2138002TXD6KSZ3V5X27, 9695009AXSRNHZE85Y20, 4R3ZURLYISNNMHMK608), and most trades use at least one of them on either the reporting or the other-counterparty side. This is the single largest reason CFTC compliance is so low. In a production setting, it would imply that upstream onboarding is treating LEI as a free-text field rather than running it through MOD 97-10 at capture time.

Second, **MAS's strictness on collateral and margin fields creates regime-specific asymmetry**. CFTC does not require collateral_portfolio_code, initial_margin_posted, or variation_margin_posted; MAS does. Several trades (T003, T004, T014, T019, T020, T023, T025) carry none of the three fields at all, and T017 carries

them as explicit null. These trades pass CFTC’s identifier and date checks but fail on MAS, producing the regime asymmetry the dashboard heatmap visualizes in red against green. Booking systems likely treat collateral fields as optional in cleared-trade workflows where the clearing house manages margin, but the reporting obligation is independent of who actually holds the collateral.

Third, **prediction contracts force a four-cell jurisdictional matrix**. T026 and T028 trade on Kalshi (a CFTC-regulated DCM) — CFTC CONDITIONAL pending the CFTC’s ANPR 91 FR 12516 (CFTC 2026); T027 trades on Polymarket via VPN bypass — CFTC NOT_APPLICABLE because the venue is offshore; all three are MAS NOT_APPLICABLE because the MAS taxonomy does not classify event contracts as OTC derivatives. T034 (author-designed) extends this pattern with a JudicialOutcome variant.

Recommendations to a compliance team would include: (i) validate LEI checksums at trade-booking time via the GLEIF live API and reject ill-formed LEIs before SDR submission; (ii) pre-flight reference-rate values against the current FPML codeset, with LIBOR-substring matches downgraded to warnings rather than errors so that legacy trades remain reportable; (iii) treat null collateral fields on cleared trades as a systemic upstream issue in the OMS rather than a reporting fix; (iv) maintain a “no UPI” review queue for prediction-contract-like trades while regulatory classification under CFTC ANPR 91 FR 12516 is finalised.

4. Classification Analysis — Prediction Contracts (Module 4)

The full Module 4 written analysis is in `src/module4_analysis.MD` is structured around four sub-tasks. In brief: the three event-contract trades T026, T027, and T028 all pass Brandes’s (2026) economic-function test — each has an identifiable actor with measurable contractual exposure to a political, macroeconomic, or regulatory outcome and would benefit from a price-discovery-providing hedging instrument. A proposed UPI schema for the political-outcome variant introduces an `EventContract` asset class with attributes that capture the event description, type, jurisdiction, oracle source, and cash-only delivery. The jurisdictional asymmetry between the CFTC and MAS/EU treatment of these contracts is regulatory arbitrage in operation: actors with US regulatory access (T026, T028 via Kalshi) benefit; EU residents are forced offshore (T027 via Polymarket VPN); and EU supervisors lose. Kalshi’s ~13 B USD monthly notional (Brandes 2026; Diercks, Katz & Wright 2026) sits entirely outside any SDR because no `EventContract` asset class exists in ANNA-DSB — a silent miss the current taxonomy cannot help with.

5. Limitations and Future Work

The engine is intentionally read-only and single-pass: it processes a static `trades.json` snapshot and produces a static compliance report. Productionising in a real, regulated

environment would require a substantial set of capabilities that the current architecture lacks.

Lifecycle events. The engine only validates `action_type`: NEW. Production reporting under CFTC Reg 45 and EMIR Refit requires the full lifecycle (MODIFY, CORRECT, CANCEL, TERMINATE, REVIVE, POSC), each of which requires prior-state retrieval from the SDR and a diff computation.

Valuation reporting. CFTC Part 45.4 and EMIR Refit Table 2 both require daily mark-to-market reporting of open positions, including margin and collateral movements. This needs pricing-engine connectivity for non-cleared derivatives and pass-through of CCP valuations for cleared ones; the engine performs no valuation.

UTI pairing. Both counterparties must report the same UTI; SDR pairing engines (DTCC GTR, KDPW, UnaVista) reconcile reports and flag mismatches. The engine validates the UTI shape from a single side but does not handle pairing, which requires the upstream UTI generation and communication protocol that production OMS handles.

Live SDR connectivity. Production submits via REST or SFTP to a registered SDR/TR, with acknowledgment, rejection, and retransmission handling. The engine writes JSON to the local disk; the last mile to a registered repository is not implemented.

Prediction-contract extensions. T026–T028 and T034 fall into `NO_PRODUCT_DEFINITION`. The §4B schema in `module4_analysis.md` proposes the canonical structure, but implementation requires an ANNA-DSB library extension, an oracle-registry codeset publication, and DCM coordination — the analytical work is here; the institutional work is not.

Reference-data freshness. Code sets drift: new currencies, deprecated rates, and new templates appear quarterly. Production engines auto-refresh against the DSB API and GLEIF concatenated file; this repository assumes a static snapshot.

Multi-jurisdictional fan-out. One trade is often reportable to two or three regimes simultaneously, each with regime-specific field transforms and timestamp conventions. The engine validates CFTC and MAS in parallel but does not generate separate per-regime submission payloads.

The engine demonstrates a complete validation surface against a static portfolio snapshot. The operational ceremony of running it as a live reporting platform — lifecycle, pairing, valuation, connectivity, freshness, and routing — remains future work.

Team Contribution Statement

Jagan Augustine (G2505229D) — Domain Expert. Jagan led the regulatory and policy work. He interpreted the field-requirement tables for CFTC Part 45 and the MAS Securities and Futures (Reporting of Derivatives Contracts) Regulations, validated the trade economics

for the three provided event contracts (T026 / T027 / T028) and the author-designed T034 JudicialOutcome variant, and authored Module 4 (`src/module4_analysis.md`) in full — including the Brandes (2026) three-question economic-function test for the three prediction-contract trades, the \$4B UPI schema proposal for the political-outcome variant, the operationalisation of the Contract Scope Limitations and Market Integrity Supervision framework elements, and the engine-limits reflection. He also drafted §1 (Regulatory Landscape) of this report, mapping the three identifier pillars (UPI / UTI / LEI) to the divergence between regimes.

Vivin Joghee (G2505378K) — Engineer. Vivin owned the architecture and engineering. He designed the five-module JSON-in / JSON-out pipeline, built each module’s CLI entry point and the Flask + plotly dashboard, and chose the matching strategy in Module 2 — exact-then-heuristic with token-overlap scoring rather than a curated alias table — which resolved the trade-vs-template use-case naming divergence without per-trade maintenance. He implemented the deterministic base32 mock UPI (hashing only UPI-defining attributes so two trades on the same product collide deterministically), maintained the repository structure and `.gitignore` decisions, vendored the 38 MB ANNA-DSB Product Definitions library to satisfy Deliverable 1’s “copy of” requirement, and authored §2 (System Architecture) of this report.

Pua Sin Yee / Clara (G2508087C) — Validation Specialist. Clara owned the validation surface. She implemented the ISO 4217 currency check via `pycountry` (including the XAU precious-metal carve-out), the ISO 7064 MOD 97-10 LEI checksum (using `python-stdnum.lei.validate` plus a hand-implemented `compute_lei_check_digits` helper retained for educational transparency), and the ISO 23897 UTI namespace/suffix validator. She designed the Module 3 per-field `{value, valid, error}` audit-trail schema that makes every check explicit in the output rather than only surfacing failures, and identified the three dataset LEIs (2138002TXD6KSZ3V5X27, 9695009AXSRNHZE85Y20, 4R3ZURLYISNNMHMK608) that fail MOD 97-10 — the assignment’s “test LEIs are not all real” teaching point. She authored §3 (Portfolio Findings) of this report.

The Researcher role — deeper analysis of the MAS jurisdictional choice, design of the six author-added trades (T029–T034) for Deliverable 2, including the two intentional errors (T031 UTI namespace mismatch, T032 unrecognized `action_type`), and shaping the dashboard’s four required visualizations — was shared among all three members.

References

Bank for International Settlements [BIS]. (2025). *Statistical release: OTC derivatives statistics at end-June 2025*. Monetary and Economic Department.

Brandes, A. (2026). *The Unhedgeable State: Political Risk, Prediction Markets, and the Gap in Europe’s Risk Management Architecture*. Policy Brief, April 2026.

Commodity Futures Trading Commission [CFTC]. (2012). *Swap Data Recordkeeping and Reporting Requirements*—17 CFR Part 45.

Commodity Futures Trading Commission [CFTC]. (2026). *Advance Notice of Proposed Rulemaking: Prediction Markets*. 91 FR 12516, RIN 3038-AF65, 16 March 2026.

Committee on Payments and Market Infrastructures and the International Organization of Securities Commissions [CPMI-IOSCO]. (2018). *Harmonization of critical OTC derivatives data elements (other than UTI and UPI) — Technical Guidance*. April 2018.

Diercks, A. M., Katz, J. D., & Wright, J. H. (2026). *Kalshi and the Rise of Macro Markets*. NBER Working Paper No. 34702 / Federal Reserve Board FEDS 2026-010.

Regulation (EU) No 648/2012 of the European Parliament and of the Council (EMIR). *Official Journal of the European Union*, L 201, 27 July 2012.

Regulation (EU) 2019/834 of the European Parliament and of the Council (EMIR Refit). *Official Journal of the European Union*, L 141, 28 May 2019.

International Organization for Standardization. ISO 4914 *Securities and related financial instruments — Unique Product Identifier (UPI)*. ISO 17442 *Financial services — Legal Entity Identifier (LEI)*. ISO 23897 *Financial services — Unique Transaction Identifier (UTI)*. ISO 7064 *Information technology — Security techniques — Check character systems*. ISO 4217 *Codes for representing currencies*.